Verified Interval Abstract Interpreter (Lean 4)

A mechanically verified interval abstract interpreter for a small imperative language. The analyzer computes sound over-approximations of variable value ranges — formally proved correct in Lean 4 with no heuristics and no false negatives.

Prerequisites

Tool	Version	How to install
elan (Lean version manager)	any	`curl https://elan.lean-lang.org/install.sh | sh`
Lean 4	`v4.26.0`	installed automatically by elan from `lean-toolchain`
lake	bundled with Lean	—
git	any	system package manager

> **macOS**: run `xcode-select --install` first if you haven't already.
> **Windows**: WSL2 recommended.

Build

```
git clone https://github.com/kiwi8803/interval_analyzer_aam.git
cd interval_analyzer_aam
lake build
```

First build: downloads the Mathlib `.olean` cache automatically — allow **~5 minutes**.
Subsequent builds: incremental, ~10–30 seconds.

Successful build ends with:

```
Build completed successfully (3069 jobs)
```

> You will see `warning: declaration uses 'sorry'` from `Fixpoint.lean` — this is expected. These warnings are in a voluntary proof extension and do **not** affect the analyzer's correctness or the ability to run examples.

Run

```
lake exe analyzer   # ~5–10 seconds
```

This runs four analysis demos and prints the inferred variable intervals.

Reading the Output

The analyzer prints intervals using Lean's internal type names. Here is the translation key:

Output	Meaning
`Analyzer.ExtendedInt.fin N`	the integer N
`Analyzer.ExtendedInt.pos_inf`	+∞
`Analyzer.ExtendedInt.neg_inf`	−∞
`Analyzer.Interval.range L H`	the interval [L, H]
`Analyzer.Interval.bot`	⊥ (unreachable / empty)

Example: `Interval.range (fin 5) (fin 15)` means **x ∈ [5, 15]**.

Expected Output

Demo 1 — Straight-line analysis

```
Program:  x := 5;  y := x + 3
```

```
[("x", Interval.range (fin 5) (fin 5)),
 ("y", Interval.range (fin 8) (fin 8))]
```

x ∈ [5, 5], y ∈ [8, 8] — exact values, no widening needed.

Demo 2 — If-else branch merging

```
Program:  (initial x ∈ [-32, 5])   if x then y := 1  else y := 0
```

```
[("x", Interval.range (fin (-32)) (fin 5)),
 ("y", Interval.range neg_inf (fin 1))]
```

x ∈ [−32, 5], y ∈ (−∞, 1] — both branches explored and joined. The analysis is sound: the true values `y ∈ {0, 1}` are contained in (−∞, 1].

Demo 3 — While loop with widening

```
Program:  x := 1;  while x do  x := x + 1
```

```
[("x", Interval.range (fin 1) pos_inf)]
```

x ∈ [1, +∞) — the upper bound is widened to +∞ after the threshold, forcing convergence. All concrete values 1, 2, 3, … are contained in [1, +∞).

Demo 4 — Security: countdown loop

```
Program:  x := 10;
          while x do  x := x - 1;
          danger := x + 5
```

```
[("danger", Interval.range (fin 5) (fin 15)),
 ("x",      Interval.range (fin 0) (fin 10))]
```

x ∈ [0, 10], danger ∈ [5, 15]
Concrete: `x = 0` after the loop, so `danger = 5` exactly.
Abstract: [5, 15] is a sound over-approximation — it contains the true value 5. Loop-analysis capping recovers tighter bounds than plain widening would give.

More Examples (Interactive)

Six demos with additional programs are available in `Analyzer/Demo.lean` as `#eval` expressions. Each runs in **under 1 second** in VS Code.

In VS Code:

1. Install the **Lean 4** extension from the marketplace.
2. Open `Analyzer/Demo.lean`.
3. Place your cursor on any `#eval` line — result appears in the **Lean Infoview** panel (`Ctrl+Shift+Enter` to open it).

In the terminal:

```
lake env lean Analyzer/Demo.lean
```

All six demos at a glance

#	Program	Expected result
1	`x := 5; y := x + 3`	`x ∈ [5,5], y ∈ [8,8]`
2	`if x then y:=1 else y:=0` (x ∈ [−32,5])	`y ∈ [0,1]`
3	`while x do x := x+1` (x starts at 1)	`x ∈ [1, +∞)`
4	Countdown: `x:=10; while x do x:=x−1; danger:=x+5`	`x ∈ [−1,10], danger ∈ [5,15]`
5	Increasing: `x:=−5; while x do x:=x+1; result:=x+10`	`x ∈ [−5,0], result ∈ [5,10]`
6	Nested loops: outer countdown + inner reset	`out ∈ (−∞,5], inner ∈ (−∞,3]`

Writing Your Own Program

Language Reference

The analyzer accepts a small imperative language with the following constructs. All types live in the `Analyzer` namespace.

Expressions (`Expr`)

Constructor	Lean syntax	Meaning
Integer literal	`.const n`	the integer `n` (use negative literals: `.const (−5)`)
Variable read	`.var "x"`	read the current value of variable `x`
Addition	`.add e1 e2`	`e1 + e2`
Subtraction	`.sub e1 e2`	`e1 − e2`

> **No multiplication or division.** The language is intentionally minimal. Extend by adding constructors to `Analyzer/Syntax.lean` and cases to `evalExpr` in `Analyzer/State.lean`.

Statements (`Stmt`)

Constructor	Lean syntax	Meaning
No-op	`.skip`	does nothing; used as a program terminator
Assignment	`.assign "x" e`	`x := e`

Constructor	Lean syntax	Meaning
Sequence	`.seq s1 s2`	`s1; s2` — execute s1 then s2
Conditional	`.ite e s1 s2`	`if e ≠ 0 then s1 else s2`
While loop	`.while e s`	`while e ≠ 0 do s`

Boolean semantics: the language uses C-style integer-as-boolean. An expression is "true" if it can be non-zero and "false" if it can be zero. When the interval overlaps both, the abstract machine explores both branches.

Initial intervals (`Interval`)

Value	Lean	Meaning
Unknown / uninitialised	`.bot`	⊥ — no value yet
Known exact value	`.range (.fin n) (.fin n)`	exactly `n`
Known range	`.range (.fin lo) (.fin hi)`	`[lo, hi]`
Lower-bounded	`.range (.fin lo) .pos_inf`	`[lo, +∞)`
Upper-bounded	`.range .neg_inf (.fin hi)`	`(−∞, hi]`
Fully unknown	`.range .neg_inf .pos_inf`	`(−∞, +∞)` — any integer

API

```
-- Build an initial configuration from a program and variable initial
values
mkConfig (prog : Stmt) (vars : List (String × Interval)) : Config

-- Run the analyzer; threshold = widening delay (higher = more precise,
slower)
widenFixpoint (threshold : Nat) (init : Config) : List Config

-- Extract variable intervals from the output
showVars (env : Env) (configs : List Config) : List (String × Interval)

-- Convenience: mkConfig + widenFixpoint threshold=1 + showVars combined
run (prog : Stmt) (init : List (String × Interval)) : List (String ×
Interval)
```

Step-by-step example

```
-- Program:  i := 0;  s := 0;  while i < 10 do { s := s + i; i := i + 1 }
-- (approximated as: while i do ... since we have no comparison operator)
```

```
def sum_loop : Analyzer.Stmt :=
  .seq (.assign "i" (.const 0))
  (.seq (.assign "s" (.const 0))
  (.seq (.while (.var "i")
          (.seq (.assign "s" (.add (.var "s") (.var "i")))
                (.assign "i" (.add (.var "i") (.const 1)))))
        .skip))   -- ← .skip required after a top-level while

#eval Analyzer.Demo.run sum_loop [("i", .bot), ("s", .bot)]
-- Expected (widening kicks in): i ∈ [0, +∞),  s ∈ [0, +∞)
```

Add this `#eval` to `Analyzer/Demo.lean` and open it in VS Code, or run:

```
lake env lean Analyzer/Demo.lean
```

To run via `lake exe analyzer`, add a runner to `Analyzer.lean`:

```
def run_sum_loop : IO Unit := do
  let res := Analyzer.Demo.run sum_loop [("i", .bot), ("s", .bot)]
  IO.println (repr res)
-- Then call run_sum_loop inside main
```

Common patterns and caveats

Sequencing multiple statements: Use nested `.seq`. The last statement should always be `.skip` if it follows a `while` (so the continuation stack has a `kSeq` frame on exit):

```
.seq stmt1 (.seq stmt2 (.seq stmt3 .skip))
```

Widening threshold:

```
widenFixpoint 1  cfg   -- widen immediately (fast, least precise)
widenFixpoint 5  cfg   -- allow 5 join steps before widening
widenFixpoint 12 cfg   -- more precise for loops with known small bounds
(see Demo 4)
```

A threshold of `1`–`3` is sufficient for most programs. Use higher values only if the loop body has complex structure that the analysis needs to explore precisely first.

Negative constants:

```
.const (-5)    -- correct: negative integer literal
.const -5      -- parse error: use parentheses
```

Variables must be declared in the initial list: Every variable the program writes to must appear in the `vars` list passed to `mkConfig` / `run`, even if its initial value is `.bot` (unknown). Variables not in the list are invisible to the environment.

No reads of undeclared variables: Reading a variable not in the initial list returns ⊥ (bot), which silently propagates through expressions. Declare all variables upfront.

Project Structure

```
interval_analyzer_aam/
├── Analyzer.lean              ← Entry point: main function, 4 demos
├── Analyzer/
│   ├── Syntax.lean            ← Language AST (Stmt, Expr)
│   ├── Domain.lean            ← Interval lattice, α, γ, widening
│   ├── State.lean             ← Abstract CESK machine (step)
│   ├── Concrete.lean          ← Concrete CESK machine (BigStep)
│   ├── GaloisConnection.lean  ← All soundness proofs — no sorry
│   ├── Fixpoint.lean          ← Fixpoint bridge extension (some sorry)
│   └── Demo.lean              ← Six #eval examples
├── lean-toolchain             ← Pins Lean 4 v4.26.0
└── lakefile.toml              ← Build config (Mathlib + Batteries)
```

What is formally proved (no sorry, no axioms)

```
GaloisConnection.lean:
  galois_connection   — α(S) ≤ i  ↔  S ⊆ γ(i)
  step_local_sound    — one concrete step has a matching abstract step
  global_soundness    — full execution is soundly over-approximated
(induction on BigStep)
  widen_sound         — i₁ ⊔ i₂  ≤  i₁ ∇ i₂
```

Timing Summary

Action	Time
First `lake build` (Mathlib cache download)	~5 min
Incremental `lake build`	~10–30 sec
`lake exe analyzer`	~5–10 sec
`#eval` in VS Code Infoview	< 1 sec per example

Troubleshooting

`lake build` fails with "unknown package"
→ Run `lake update` then `lake build` again.

Lean extension shows errors everywhere on first open
→ Wait for indexing to complete (progress in the VS Code status bar, ~3–5 min).

`warning: declaration uses 'sorry'`
→ Expected. These are in `Fixpoint.lean` (a voluntary proof extension). Ignore them.

`lake exe analyzer` produces no output
→ Ensure `lake build` completed successfully first.